# Lists, S3, & Python Classes

## Lecture 05

Dr. Colin Rundel

# R Lists

# Lists

Lists are R's other vector type (generic). Unlike atomic vectors they are *heterogeneous* - each element can be any R object: atomic vectors, other lists, functions, etc.

```r
list("A", (1:4)/2, list(1L), sum)
```

```
[[1]]
[1] "A"

[[2]]
[1] 0.5 1.0 1.5 2.0

[[3]]
[[3]][[1]]
[1] 1


[[4]]
function (..., na.rm = FALSE)  .Primitive("sum")
```

```python
["A", [0.5, 1.0, 1.5, 2.0], [1], sum]
```

```
['A', [0.5, 1.0, 1.5, 2.0], [1], <built-in funct
```

# List structure

The printed form of a list is verbose. `str()` gives a compact summary of any R object's structure and is particularly useful for lists.

```r
str(c(1, 2))
```

```
num [1:2] 1 2
```

```r
str(1:100)
```

```
int [1:100] 1 2 3 4 5 6 7 8 9 10 ...
```

```r
str("A")
```

```
chr "A"
```

```r
str( list(
  "A", c(TRUE, FALSE),
  (1:4)/2, list(TRUE, 1),
  function(x) x^2
) )
```

```
List of 5
 $ : chr "A"
 $ : logi [1:2] TRUE FALSE
 $ : num [1:4] 0.5 1 1.5 2
 $ :List of 2
  ..$ : logi TRUE
  ..$ : num 1
 $ :function (x)
```

# Nested lists

Lists can contain other lists, so they do not have to be flat.

This makes them a natural way of representing tree-like data (e.g. JSON).

```r
x = list(1, list(2, list(3, 4), 5))
str(x)
```

```
List of 2
 $ : num 1
 $ :List of 3
  ..$ : num 2
  ..$ :List of 2
  .. ..$ : num 3
  .. ..$ : num 4
  ..$ : num 5
```

```python
x = [1, [2, [3, 4], 5]]
x
```

```
[1, [2, [3, 4], 5]]
```

# Named lists

Elements of a list (or atomic vector) can be named. Names help avoid magic numbers when accessing elements (more readable code).

A valid name starts with a letter or `.` (not followed by a digit) and contains only `[A-Za-z0-9._]`.

```r
x = list(A = 1, B = list(C = 2, D = 3))
str(x)
```

```
List of 2
 $ A: num 1
 $ B:List of 2
  ..$ C: num 2
  ..$ D: num 3
```

```r
names(x)
```

```
[1] "A" "B"
```

Any other name must be surrounded with backticks,

```r
list("knock knock" = "who's there?")
```

```
$`knock knock`
[1] "who's there?"
```

# [ vs [[

R has two addition subsetting operators.

- `[[` extracts a *single element*
- `$` is shorthand for named lookup with `[[`

```r
y = list(a = 1, b = 4, c = 7:9)
```

```r
y[2]
```

```
$b
[1] 4
```

```r
str( y[2] )
```

```
List of 1
 $ b: num 4
```

```r
y["b"]
```

```
$b
[1] 4
```

```r
str( y["b"] )
```

```
List of 1
 $ b: num 4
```

```r
y[[2]]
```

```
[1] 4
```

```r
str( y[[2]] )
```

```
 num 4
```

```r
y[["b"]]
```

```
[1] 4
```

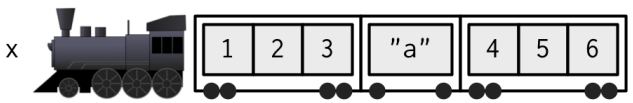
```r
str( y[["b"]] )
```

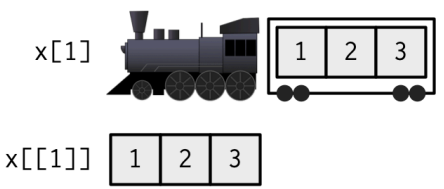
```
 num 4
```

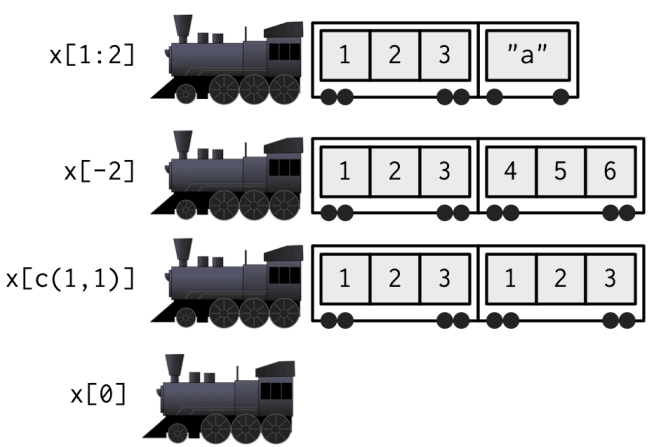
```r
y$b
```

```
[1] 4
```

# Hadley's analogy



# [[ details

`[[` selects by position or by name and only ever returns one element.

```r
y = list(a = 1, b = 4, c = 7:9)
```

```r
y[[3]]
```

```
[1] 7 8 9
```

```r
y[["c"]]
```

```
[1] 7 8 9
```

```r
y[[4]]
```

```
Error in `y[[4]]`:
! subscript out of bounds
```

```r
y[["d"]]
```

```
NULL
```

Vectors of length > 1 are interpreted as *recursive* indexing - `y[[c(3, 2)]]` is `y[[3]][[2]]`.

```r
y[[c(3, 2)]]
```

```
[1] 8
```

```r
y[[1:2]]
```

```
Error in `y[[1:2]]`:
! subscript out of bounds
```

# $ subsetting

`$` is a shorthand for `[[` with a *literal* name, so `y$c` is equivalent to `y[["c"]]`. It only works with lists and objects built using lists (e.g.data frames) and it *partially matches names*.

```R
y = list(abc = 1, def = 5)
```

```R
y[["abc"]]
```

```
[1] 1
```

```R
y$abc
```

```
[1] 1
```

```R
y$a
```

```
[1] 1
```

```R
y[["a"]]
```

```
NULL
```

```R
x = c(abc = 1, def = 5)
```

```R
x[["abc"]]
```

```
[1] 1
```

```R
x$abc
```

```
Error in `x$abc`:
! $ operator is invalid for atomic vectors
```

A common error is using `$` with a variable that holds a name,

# Modifying lists

Assignment with `[[` or `$` replaces an element or adds a new one.

Assigning `NULL` removes an element.

R

```r
x = list(a = 1, b = 2)
x$c = "new"
x[["a"]] = 100
str(x)
```

```
List of 3
 $ a: num 100
 $ b: num 2
 $ c: chr "new"
```

R

```r
x$b = NULL
str(x)
```

```
List of 2
 $ a: num 100
 $ c: chr "new"
```

py

```python
x = [1, 2]
x.append("new")
x[0] = 100
x
```

```
[100, 2, 'new']
```

py

```python
del x[1]
x
```

```
[100, 'new']
```

# Lists and atomic vectors

Combining an atomic vector with a list via `c()` produces a list (the more generic type).

`unlist()` goes the other way, flattening a list into an atomic vector with the usual coercion rules.

```r
str( c(1, list(4, list(6, 7))) )
```

```
List of 3
 $ : num 1
 $ : num 4
 $ :List of 2
  ..$ : num 6
  ..$ : num 7
```

```r
unlist( list(1:3, list(4:5, 6)) )
```

```
[1] 1 2 3 4 5 6
```

```r
unlist( list(1, list(2, list(3, "Hello"))) )
```

```
[1] "1"     "2"     "3"     "Hello"
```

# Exercise 1

Represent the following JSON data as a list in R (we will revisit this in Python next week).

```
{
  "firstName": "John",
  "lastName": "Smith",
  "age": 25,
  "address":
  {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": 10021
  },
  "phoneNumber":
  [ {
      "type": "home",
      "number": "212 555-1239"
    },
    {
      "type": "fax",
      "number": "646 555-4567"
  } ]
}
```

Once you have the list:

- Extract the fax number using `$` and `[[`.
- Add a `"mobile"` phone number.
- Remove the `age` element.

# Attributes

# Attributes

Attributes are metadata attached to an R object. Some attributes are special (e.g. `names`, `dim`, `dimnames`, `class`, `levels`, etc.) because they modoify how the object behaves / is treated.

Attributes are stored as a *named list* attached to the object, accessed via `attributes()` or `attr()`.

```r
(x = c(L = 1, M = 2, N = 3))
```

```
L M N
1 2 3
```

```r
str( attributes(x) )
```

```
List of 1
 $ names: chr [1:3] "L" "M" "N"
```

```r
attr(x, "names")
```

```
[1] "L" "M" "N"
```

```r
attr(x, "other")
```

```
NULL
```

# Setting attributes

Most important attributes have helper functions for getting and setting (`names()`, `dim()`, `class()`, `levels()`),

```r
names(x) = c("Z", "Y", "X")
x
```

```
Z Y X
1 2 3
```

```r
names(x) = 1:3
x
```

```
1 2 3
1 2 3
```

```r
attributes(x)
```

```
$names
[1] "1" "2" "3"
```

```r
attr(x, "other") = "anything"
x
```

```
1 2 3
1 2 3
attr(,"other")
[1] "anything"
```

```r
str( attributes(x) )
```

```
List of 2
 $ names: chr [1:3] "1" "2" "3"
 $ other: chr "anything"
```

Note that `names(x) = 1:3` silently coerced the integers to character, since the `names` attribute must be a character vector

# Factors

Factors are how R represents categorical data - a variable with a discrete set of possible values (called levels).

```r
(x = factor(c("Sunny", "Cloudy", "Rainy", "Cloudy", "Cloudy")))
```

```
[1] Sunny  Cloudy Rainy  Cloudy Cloudy
Levels: Cloudy Rainy Sunny
```

```r
str(x)
```

```
 Factor w/ 3 levels "Cloudy","Rainy",..: 3 1 2 1 1
```

```r
typeof(x)
```

```
[1] "integer"
```

```r
class(x)
```

```
[1] "factor"
```

```r
mode(x)
```

```
[1] "numeric"
```

```r
levels(x)
```

```
[1] "Cloudy" "Rainy"  "Sunny"
```

# Composition

A factor is just an integer vector with two attributes: `levels` and `class`.

```r
str( attributes(x) )
```

```
List of 2
 $ levels: chr [1:3] "Cloudy" "Rainy" "Sunny"
 $ class : chr "factor"
```

```r
unclass(x)
```

```
[1] 3 1 2 1 1
attr(,"levels")
[1] "Cloudy" "Rainy"  "Sunny"
```

We can build our own from scratch using `attr()`,

```r
y = c(3L, 1L, 2L, 1L, 1L)
attr(y, "levels") = c("Cloudy", "Rainy", "Sunny")
attr(y, "class") = "factor"
y
```

```
[1] Sunny  Cloudy Rainy  Cloudy Cloudy
Levels: Cloudy Rainy Sunny
```

# Building objects with `structure()`

Setting attributes one at a time is clunky - `structure()` attaches any number of attributes to an object in a single call and is the standard way to construct objects like this.

```r
( y = structure(
    c(3L, 1L, 2L, 1L, 1L),
    levels = c("Cloudy", "Rainy", "Sunny"),
    class = "factor"
) )
```

```
[1] Sunny  Cloudy Rainy  Cloudy Cloudy
Levels: Cloudy Rainy Sunny
```

```r
class(y)
```

```
[1] "factor"
```

```r
is.factor(y)
```

```
[1] TRUE
```

```r
identical(x, y)
```

```
[1] TRUE
```

# Factors are integer vectors?

Knowing that factors are stored as integers explains some of their more surprising behaviors,

```r
x + 1
```

```
Warning in Ops.factor(x, 1): '+' not meani
[1] NA NA NA NA NA
```

```r
is.integer(x)
```

```
[1] FALSE
```

```r
is.numeric(x)
```

```
[1] FALSE
```

```r
as.numeric(factor(c("10", "20")))
```

```
[1] 1 2
```

```r
as.integer(x)
```

```
[1] 3 1 2 1 1
```

```r
as.character(x)
```

```
[1] "Sunny"  "Cloudy" "Rainy"  "Cloudy" "C
```

```r
as.logical(x)
```

```
[1] NA NA NA NA NA
```

```r
as.numeric(
  as.character(factor(c("10", "20")))
)
```

```
[1] 10 20
```

# S3 Object System

# `class`

The `class` attribute adds a layer on top of R's type hierarchy - previously we saw `typeof()` and `mode()`.

| value | `typeof()` | `mode()` | `class()` |
|---|---|---|---|
| `TRUE` | logical | logical | logical |
| `1` | double | numeric | numeric |
| `1L` | integer | numeric | integer |
| `"A"` | character | character | character |
| `NULL` | NULL | NULL | NULL |
| `list(1, "A")` | list | list | list |
| `factor("A")` | integer | numeric | factor |
| `matrix(1:4, 2)` | integer | numeric | matrix, array |
| `function(x) x^2` | closure | function | function |
| `sum` | builtin | function | function |

# S3 class specialization

```r
x = c("A", "B", "A", "C")
```

```r
print( x )
```

```
[1] "A" "B" "A" "C"
```

```r
print( factor(x) )
```

```
[1] A B A C
Levels: A B C
```

```r
print( unclass( factor(x) ) )
```

```
[1] 1 2 1 3
attr(,"levels")
[1] "A" "B" "C"
```

```r
print.default( factor(x) )
```

```
[1] 1 2 1 3
```

# What's up with `print`?

```R
print
```

```
function (x, ...)
UseMethod("print")
<bytecode: 0x769f953e38>
<environment: namespace:base>
```

`print` does no printing itself - `UseMethod()` looks at the class of `x` and calls the matching *method*. For a factor that is `print.factor()`, for everything without a more specific method `print.default()` is used.

```
1 print.default
```

```
function (x, digits = NULL, quote = TRUE, na.pri
    right = FALSE, max = NULL, width = NULL, use
    ...)
{
    args <- pairlist(digits = digits, quote = qu
        print.gap = print.gap, right = right, ma
        useSource = useSource, ...)
    missings <- c(missing(digits), missing(quote
        missing(print.gap), missing(right), miss
        missing(useSource))
    .Internal(print.default(x, args, missings))
}
<bytecode: 0x769ca2cba0>
<environment: namespace:base>
```

```
1 print.factor
```

```
function (x, quote = FALSE, max.levels = NULL, w
    ...)
{
    ord <- is.ordered(x)
    if (length(x) == 0L)
        cat(if (ord)
            "ordered"
        else "factor", "()\n", sep = "")
    else {
        xx <- character(length(x))
        xx[] <- as.character(x)
        keepAttrs <- setdiff(names(attributes(x)
            "class"))
        attributes(xx)[keepAttrs] <- attributes(
        print(xx, quote = quote, ...)
    }
    maxl <- max.levels %||% TRUE
    if (maxl) {
        n <- length(lev <- encodeString(levels(x
            "\"", "")))
        colsep <- if (ord)
            " < "
        else " "
```

# Generics are everywhere

Many of the base R functions you use regularly are S3 generics whose only job is to dispatch on class,

```r
mean
```

```
function (x, ...)
UseMethod("mean")
<bytecode: 0x769e0b5000>
<environment: namespace:base>
```

```r
plot
```

```
function (x, y, ...)
UseMethod("plot")
<bytecode: 0x7696ffa510>
<environment: namespace:base>
```

```r
summary
```

```
function (object, ...)
UseMethod("summary")
<bytecode: 0x7698211540>
<environment: namespace:base>
```

```r
t.test
```

```
function (x, ...)
UseMethod("t.test")
<bytecode: 0x769f0dca18>
<environment: namespace:stats>
```

Other generics, such as `sum`, dispatch without an explicit call to `UseMethod()`:

```r
sum
```

```
function (..., na.rm = FALSE)  .Primitive("sum")
```

`sum` is a primitive implemented in C. It still performs S3 dispatch internally, rather than calling `UseMethod()` in its visible

# What is S3?

> S3 is R's first and simplest OO system. S3 is R's first and simplest OO system. S3 is informal and ad hoc, but there is a certain elegance in its minimalism: you can't take away any part of it and still have a useful OO system.
>
> - Hadley Wickham, Advanced R

S3 is the only OO system used in the base and stats packages and the most common on CRAN. It should not be confused

# Dispatch

S3 dispatch is very simple - a generic calls `UseMethod("func")`, which searches for a function named `<func>.<class>`, trying each class of the first argument in order and falling back to `<func>.default` if none is found.

`methods()` lists the methods available for a generic, or all the methods available for a class,

```r
methods("summary")
```

```
 [1] summary,ANY-method                    summary,diagonalMatrix-method
 [3] summary,sparseMatrix-method           summary.aov
 [5] summary.aovlist*                      summary.aspell*
 [7] summary.check_packages_in_dir*        summary.connection
 [9] summary.data.frame                    summary.Date
[11] summary.default                       summary.difftime
[13] summary.ecdf*                         summary.factor
[15] summary.free1way*                     summary.glm
[17] summary.infl*                         summary.lm
[19] summary.loess*                        summary.manova
[21] summary.matrix                        summary.mlm*
[23] summary.nls*                          summary.packageStatus*
[25] summary.pandas.core.frame.DataFrame*  summary.pandas.core.series.Series*
[27] summary.pandas.DataFrame*             summary.pandas.Series*
[29] summary.POSIXct                       summary.POSIXlt
[31] summary.ppr*                          summary.prcomp*
[33] summary.princomp*                     summary.proc_time
```

```
[35] summary.python.builtin.object*      summary.rlang_error*
[37] summary.rlang_message*              summary.rlang_trace*
[39] summary.rlang_warning*              summary.rlang:::list_of_conditions*
[41] summary.shingle*                    summary.srcfile
[43] summary.srcref                      summary.stepfun
[45] summary.stl*                        summary.table
```

```r
methods(class = "factor")
```

```
 [1] [            [[           [[<-         [<-          all.equal
 [6] Arith        as.character as.data.frame as.Date     as.list
[11] as.logical   as.POSIXlt   as.vector     c            cbind2
[16] coerce       Compare      droplevels    format       free1way
[21] initialize   is.na<-      kronecker     length<-     levels<-
[26] Logic        Math         Ops           plot         print
[31] rbind2       relevel      relist        rep          show
[36] slotsFromS3  summary      Summary       xtfrm
see '?methods' for accessing help and source code
```

# Adding methods

Because dispatch is by *name*, adding a method for a new class just requires defining a new function with the proper name - no registration is needed.

```r
( x = structure(
    c(1, 2, 3),
    class = "class_A") )
```

```
[1] 1 2 3
attr(,"class")
[1] "class_A"
```

```r
( y = structure(
    c(6, 5, 4),
    class = "class_B") )
```

```
[1] 6 5 4
attr(,"class")
[1] "class_B"
```

```r
print.class_A = function(x, ...) {
  cat("(Class A) ")
  print.default(unclass(x))
}
```

```r
print.class_B = function(x, ...) {
  cat("(Class B) ")
  print.default(unclass(x))
}
```

```r
print(x)
```

```
(Class A) [1] 1 2 3
```

```r
print(y)
```

```
(Class B) [1] 6 5 4
```

```r
class(x) = "class_B"
print(x)
```

```
(Class B) [1] 1 2 3
```

```r
class(y) = "class_A"
print(y)
```

```
(Class A) [1] 6 5 4
```

# Writing a good `print` method

In general, methods should match the generic's signature (`x, ...`). For print you should then manage output via `cat()` or `print()`, and return the input *invisibly* so that `print(x)` can be used in a pipeline.

```r
pct = structure(c(0.12, 0.5, 1), class = "percent")

print.percent = function(x, ...) {
  print(paste0(unclass(x) * 100, "%"), quote = FALSE)
  invisible(x)
}
```

```r
pct
```

```
[1] 12%  50%  100%
```

```r
unclass(z)
```

```
[1] 0.12 0.50 1.00
```

```r
z = print(pct)
```

```
[1] 12%  50%  100%
```

This goes for any new method - match the signature and conform to the common expectations of the other method

# Defining a new generic

A generic is just a function that calls `UseMethod()`. By convention the first argument is the object dispatched on and `...` is included so that methods can add arguments.

```R
shuffle = function(x, ...) {
  UseMethod("shuffle")
}
```

```R
shuffle.default = function(x, ...) {
  stop("Class ", class(x), " is not supported by shuffle.", call. = FALSE)
}
```

```R
shuffle.factor = function(x, ...) {
  factor( sample(as.character(x)), levels = sample(levels(x)) )
}
```

```R
shuffle.integer = function(x, ...) {
  sample(x)
}
```

# Shuffle results

```r
shuffle( 1:10 )
```

```
 [1]  4  5  6  7  1  2  9 10  8  3
```

```r
shuffle( factor(c("A", "B", "C", "A")) )
```

```
[1] A B C A
Levels: B C A
```

```r
shuffle( c(1, 2, 3, 4, 5) )
```

```
Error:
! Class numeric is not supported by shuffle.
```

```r
shuffle( letters[1:5] )
```

```
Error:
! Class character is not supported by shuffle.
```

```r
methods("shuffle")
```

```
[1] shuffle.default shuffle.factor  shuffle.integer
see '?methods' for accessing help and source code
```

# Implicit classes

Objects without a `class` attribute dispatch on an *implicit* class vector that is more detailed than what `class()` reports.

```r
report = function(x) {
  UseMethod("report")
}
report.default = function(x) paste0("Class ", class(x), " does not have a method defined.")
report.integer = function(x) "I'm an integer!"
report.double  = function(x) "I'm a double!"
report.numeric = function(x) "I'm a numeric!"
```

```r
report(1)
```

```
[1] "I'm a double!"
```

```r
report(1L)
```

```
[1] "I'm an integer!"
```

```r
report("1")
```

```
[1] "Class character does not h
```

```r
rm(report.integer, report.
report(1)
```

```
[1] "I'm a numeric!"
```

```r
report(1L)
```

```
[1] "I'm a numeric!"
```

```r
rm(report.numeric)
report(1)
```

```
[1] "Class numeric does not hav
```

```r
report(1L)
```

```
[1] "Class integer does not hav
```

# Why?

From `UseMethod`'s R documentation:

> If the object does not have a class attribute, it has an implicit class. Matrices and arrays have class "matrix" or "array" followed by the class of the underlying vector. Most vectors have class the result of `mode(x)`, except that integer vectors have class `c("integer", "numeric")` and real vectors have class `c("double", "numeric")`.

The implicit class vector can be inspected with `.class2()`,

```r
.class2(1)
```

```
[1] "double"  "numeric"
```

```r
.class2(1L)
```

```
[1] "integer" "numeric"
```

```r
.class2(matrix(1:4, 2))
```

```
[1] "matrix"  "array"   "integer" "numeric"
```

`report(1)` tries `report.double`, then `report.numeric`, then `report.default` - which is why a method for `double` is found

# Python classes

# Basic syntax

A `class` statement bundles *attributes* (data) and *methods* (functions) into a new type. Methods receive the instance as their first argument, conventionally named `self`.

```py
class rect:
  """An object representing a rectangle"""
  p1 = (0, 0)
  p2 = (1, 2)

  def area(self):
    return abs((self.p1[0] - self.p2[0]) *
               (self.p1[1] - self.p2[1]))

  def set_p1(self, p1):
    self.p1 = p1
```

```py
x = rect()
x.area()
```

```
2
```

```py
x.p1
```

```
(0, 0)
```

```py
x.set_p1((1, 1))
x.area()
```

```
0
```

```py
x.p2 = (3, 3)
x.area()
```

```
4
```

# `__init__`

Calling the class (`rect()`) creates a new *instance* and then calls its `__init__()` method with any arguments - this is the constructor, and where instance attributes should be set.

```py
class rect:
  """An object representing a rectangle"""

  def __init__(self, p1=(0, 0), p2=(1, 1)):
    self.p1 = p1
    self.p2 = p2

  def area(self):
    return abs((self.p1[0] - self.p2[0]) *
               (self.p1[1] - self.p2[1]))
```

```py
rect().area()
```

```
1
```

```py
rect((0, 0), (3, 3)).area()
```

```
9
```

```py
z = rect(p1=(-1, -1))
z.p1
```

```
(-1, -1)
```

```py
z.p2
```

```
(1, 1)
```

Methods whose names start and end with double underscores (`__init__`, `__repr__`, …) are called “dunder” or *special*

# Class vs instance attributes

Attributes assigned in the class body are *class attributes* shared by every instance (useful for constants), while attributes assigned via `self` are *instance attributes* belonging to one object.

```py
class die:
  sides = 6

  def __init__(self, value=1):
    self.value = value
```

```py
d = die(3)
d.value
```

```
3
```

```py
d.sides
```

```
6
```

```py
die.sides
```

```
6
```

```py
die.value
```

```
AttributeError: type object 'die' has no attribu
```

```py
vars(d)
```

```
{'value': 3}
```

```py
[m for m in dir(d) if not m.startswith("__
```

```
['sides', 'value']
```

`dir()` lists all attributes and methods of an object (including inherited dunder methods), while `vars()` returns just the

# Mutate or return?

Python methods that mutate an object conventionally modify `self` in place and return `None`. In R, modifying an ordinary vector or list inside a function does not modify the caller's object, so a function must return the updated value.

Returning `self` from a Python mutating method instead enables method chaining.

```py
x = [3, 1, 2]
print( x.sort() )
```

```
None
```

```py
x
```

```
[1, 2, 3]
```

```py
def set_p1(self, p1):
  self.p1 = p1
  return self

rect.set_p1 = set_p1
```

```py
r1 = rect()
r2 = r1
r1.set_p1((-1, -1)).area()
```

```
4
```

```py
r2.p1
```

```
(-1, -1)
```

```py
( rect()
  .set_p1((-1, -1))
  .area()
)
```

```
4
```

# `__str__` and `__repr__`

Every object has a default string representation, but it is not usually very useful,

```py
rect()
```

```
<__main__.rect object at 0x11443ec30>
```

```py
print(rect())
```

```
<__main__.rect object at 0x114478160>
```

`__repr__()` should return an unambiguous representation, ideally valid Python that recreates the object. `__str__()` is used by `print()` and `str()`, and falls back to `__repr__()` if not defined.

```py
def rect_repr(self):
  return f"rect({self.p1}, {self.p2})"

rect.__repr__ = rect_repr
```

```py
rect()
```

```
rect((0, 0), (1, 1))
```

```py
print(rect())
```

```
rect((0, 0), (1, 1))
```

```py
[rect(), rect((1, 1), (2, 2))]
```

```
[rect((0, 0), (1, 1)), rect((1, 1), (2, 2))]
```

# Other special methods

Operators and built-in functions also dispatch to special methods - this is the mechanism behind the operator overloading we saw in Lecture 2 (`"abc" * 3`, `[1, 2] + [3, 4]`).

```py
def rect_eq(self, other):
  return self.p1 == other.p1 and self.p2 ==

def rect_contains(self, pt):
  return all(min(a, b) <= x <= max(a, b)
             for a, b, x in zip(self.p1, se

rect.__eq__ = rect_eq
rect.__contains__ = rect_contains
```

```py
rect() == rect()
```

```
True
```

```py
rect() == rect((1, 1), (2, 2))
```

```
False
```

```py
(0.5, 0.5) in rect()
```

```
True
```

```py
(2, 2) in rect()
```

```
False
```

Without `__eq__`, `==` compares object identity (like `is`). See the data model docs for the full list, including `__len__`,

# Inheritance

A class can *inherit* from another class, gaining all of its attributes and methods. The subclass can add new methods and override existing ones - `super()` gives access to the parent's version.

```py
class square(rect):
  def __init__(self, p1=(0, 0), l=1):
    if not isinstance(l, (int, float)):
      raise TypeError("l must be a number")
    self.l = l
    super().__init__(p1, (p1[0] + l, p1[1] + l))

  def __repr__(self):
    return f"square({self.p1}, {self.l})"
```

```py
s = square((1, 1), 2)
s
```

```
square((1, 1), 2)
```

```py
s.p2
```

```
(3, 3)
```

```py
s.area()
```

```
4
```

```py
(2, 2) in s
```

```
True
```

```py
square(l="a")
```

```
TypeError: l must be a number
```

# `isinstance()` vs `type()`

`isinstance()` respects inheritance while `type()` reports the exact class - the former is almost always what you want when validating inputs, but be aware of what a type inherits from.

```py
isinstance(s, square)
```

```
True
```

```py
isinstance(s, rect)
```

```
True
```

```py
type(s) is rect
```

```
False
```

```py
isinstance(3, (int, float))
```

```
True
```

```py
isinstance("3", (int, float))
```

```
False
```

```py
isinstance(True, int)
```

```
True
```

```py
type(True) is int
```

```
False
```

```py
bool.__mro__
```

```
(<class 'bool'>, <class 'int'>, <class 'object'>
```

`__mro__` (method resolution order) is the tuple of classes Python searches for inherited attributes and methods - roughly analogous to the dispatch order in an R S3 class vector. Since `bool` inherits from `int`, code accepting `int`s accepts `True`

# Comparing R & Python

# Object systems summary

| Concept | R (S3) | Python |
|---|---|---|
| define a class | `structure(list(...), class = "x")` | `class x:` with `__init__()` |
| where methods live | separate functions `generic.class()` | inside the class definition |
| dispatch | `UseMethod()` on the first argument's class | `obj.method()` on the object's type |
| printing | `print.x()` (returns `invisible(x)`) | `__repr__()` and `__str__()` |
| operators | `Ops.x()`, `` `+.x`() `` | `__add__()`, `__eq__()`, … |
| inheritance | `class = c("child", "parent")` | `class child(parent):` |
| call the parent method | `NextMethod()` | `super().method()` |
| type check | `inherits(x, "class")` | `isinstance(x, cls)` |
| list methods | `methods(class = "x")` | `dir(x)` |

# Takeaways

- R lists are heterogeneous vectors - `[` returns a list, `[[` and `$` return a single element, and assigning `NULL` removes one.
- Attributes are metadata attached to any R object, `structure()` sets them in one call, and `class` is the attribute S3 dispatches on.
- An S3 generic is a function that calls `UseMethod()`, methods are just functions named `generic.class`, and `default` catches everything else.
- Objects without a `class` attribute dispatch on an implicit class such as `c("double", "numeric")`.
- Python classes bundle data and methods, `__init__()` constructs, `__repr__()` / `__str__()` display, and `isinstance()` checks type with inheritance.
- R functions commonly return updated values; Python mutating methods commonly update objects in place - know which convention an interface follows.